



C++



Desde C++11 el lenguaje incorpora `<thread>` y todo un ecosistema moderno para trabajar con concurrencia: hebras, cerrojos, condiciones...

Y desde **C++20** se añaden **semáforos** y **barreras** como en las bibliotecas de lenguajes de más alto nivel.

Hilos

Crear un hilo es tan simple como construir un `std::thread`:

```
std::thread t(funcion, argumentos...);
```

El hilo **empieza automáticamente** al construirse.

C++ NO tiene un método `start()` como Java.

También podemos usar lambdas:

```
std::thread t([]{  
    // código concurrente  
});
```

Métodos útiles:

- `t.join()`

El hilo actual **espera** a que `t` termine. Es bloqueante.

- `t.detach()`

Se separa del hilo: el hilo sigue por su cuenta sin posibilidad de sincronización posterior.

- `std::this_thread::sleep_for(...)`

Suspende el hilo.

- `std::this_thread::yield()`

Sugiere al planificador que otro hilo puede ejecutarse. **No garantiza nada.**

💡 Igual que en Java, **C++ no garantiza prioridades reales**. El planificador depende del sistema operativo.

Mecanismos de Exclusión Mutua

C++ ofrece varias herramientas "reales" para controlar acceso a recursos compartidos.

Cerrosjos

Normales

`std::mutex`

Operaciones:

```
m.lock();  
m.unlock();
```

Delimitan la sección crítica.

No es reentrante → un mismo hilo que intente hacer `lock()` dos veces queda bloqueado para siempre.

Ejemplo:

```
std::mutex m;  
int contador = 0;  
  
void inc() {  
    m.lock();  
    ++contador;  
    m.unlock();  
}
```

Reentrantes

`std::recursive_mutex`

Permite que **el mismo hilo** adquiera el cerrojo varias veces sin bloquearse.

Muy útil en funciones recursivas o cuando un método protegido llama a otro método también protegido.

Ejemplo:

```
std::recursive_mutex rm;

void f() {
    rm.lock();
    // ...
    f(); // recursión: válido
    rm.unlock();
}
```

Objetos Atómicos

`std::atomic<T>`

No son cerrojos, pero permiten **operaciones indivisibles** sobre variables compartidas:

```
std::atomic<int> x{0};
x.fetch_add(1);
```

Son perfectos para counters, flags y estructuras lock-free.

Pero **no sustituyen a un monitor** ni permiten sincronización compleja.

Semáforos

C++20 incorpora dos tipos:

```
#include <semaphore>
```

```
std::binary_semaphore sem(1); // como un mutex
std::counting_semaphore<10> semN(3); // general
```

Métodos:

- `sem.acquire()` → `wait()`
- `sem.release()` → `signal()`

💡 Igual que en teoría:

- Si el valor inicial ≤ 1 → puede usarse como EM.
- Si puede subir por encima de 1 → **NO garantiza exclusión mutua.**

Simular un semáforo antes de C++20

Podemos simular un semáforo con:

- `std::mutex`
- `std::condition_variable`
- y una variable entera compartida

Ejemplo breve:

```
int S = 1;
std::mutex m;
std::condition_variable cv;

void wait() {
    std::unique_lock<std::mutex> lk(m);
    while (S == 0)
        cv.wait(lk);
    S--;
}

void signal() {
    std::unique_lock<std::mutex> lk(m);
    S++;
}
```

```
    cv.notify_one();  
}
```

Barreras

C++20 incluye:

- `std::barrier` → **reutilizable**
- `std::latch` → **solo una vez**, estilo "countdown"

Ejemplo de `std::barrier` :

```
std::barrier sync_point(3);  
  
void tarea() {  
    // fase 1  
    sync_point.arrive_and_wait();  
    // fase 2  
}
```

Simular una barrera antes de C++20

Con:

- 1 mutex
- 1 condition_variable
- 2 contadores

```
#include <mutex>  
#include <condition_variable>  
  
class Barrier {  
public:  
    explicit Barrier(unsigned int numThreads)  
        : numThreads_(numThreads),  
          count_(0),  
          generation_(0) {}  
};
```

```

void wait() {
    std::unique_lock<std::mutex> lock(m_);

    // Guardamos la "generación" actual
    unsigned int gen = generation_;

    // Este hilo ha llegado a la barrera
    count_++;

    if (count_ == numThreads_) {
        // Último en llegar:
        // - resetea el contador
        // - avanza la generación
        count_ = 0;
        generation_++;
        cv_.notify_all(); // despierta a todos los hilos
    } else {
        // Mientras siga en la misma generación, espera
        cv_.wait(lock, [&] {
            return gen != generation_;
        });
    }
}

private:
    std::mutex m_;           // 1 mutex
    std::condition_variable cv_; // 1 condición
    unsigned int numThreads_; // nº de hebras que deben llegar
    unsigned int count_;      // contador de hebras llegadas
    unsigned int generation_; // "ronda" actual (segunda variable)
};

```

Monitores

C++ **NO TIENE MONITORES NATIVOS** (no existe una palabra reservada tipo `monitor`).

Pero **sí podemos simular su comportamiento**. Seguimos sin poder dar una implementación fiel a la teoría porque en C++ la señalización es **señalar y continuar**, y esto **NO SE PUEDE CAMBIAR**, ya que no podemos garantizar prioridades.

Un monitor en C++ consiste en:

- **Variables compartidas** encapsuladas en una clase
- **1 mutex**
- **Una o varias condition_variables**

Cada condición tiene **su propia cola**, como en el monitor teórico.

Ejemplo:

```
class BoundedBuffer {
private:
    std::mutex m;
    std::condition_variable not_full;
    std::condition_variable not_empty;

    std::queue<int> q;
    const size_t cap;

public:
    BoundedBuffer(size_t c) : cap(c) {}

    void put(int x) {
        std::unique_lock<std::mutex> lk(m);
        while (q.size() == cap)
            not_full.wait(lk);

        q.push(x);
        not_empty.notify_one();
    }
}
```

```

int get() {
    std::unique_lock<std::mutex> lk(m);
    while (q.empty())
        not_empty.wait(lk);

    int v = q.front();
    q.pop();
    not_full.notify_one();
    return v;
}
};

```

Variables de condición

`std::condition_variable`

Son la herramienta fundamental para sincronizar hilos por condiciones lógicas.

Métodos:

- `wait(lk)`

Libera el mutex y bloquea.

- `wait(lk, pred)`

Espera hasta que `pred()` sea true (garantiza while interno).

- `notify_one()`

- `notify_all()`

Funcionan exactamente como en Java alto nivel.